Compiler Ch1–3｜上機考速查（列印版）

用途：上機寫 code 時查骨架、陷阱與各題改法；DFA／PREDICT 表用來核對程式輸出。完整題意、全部測例與改前／改後對照在 `Compiler-Ch1-Ch3-Detail.html`；每題的 README 在 `Complier/Code/NN-…/`。行號依 2026-10-15 的 .cpp；位移時搜尋關鍵字。

1｜看題 → 選檔

題目要的輸出	資料夾／檔案	重點
HTML 只判合法	01 `html_check.cpp`	VALID／INVALID；stack 配對；void、引號、註解、raw text
HTML token／tree／text	11 `html_parser.cpp`	`--tokens --tree --text`，預設 `--check`
HTML × ParserTest（印 token）	21 `html_tags.cpp`	OPEN／CLOSE／SELF／TEXT；遞迴 element 比對名稱
算式求值／postfix／AST（每行一案）	02 `expression.cpp`	改第 12 行 `MODE`：eval／postfix／ast
深巢狀算式	12 `expression.cpp`	迭代版；`--postfix`（預設）`--ast --eval`
算式 Parser 印 token／invalid input	18 `expr_parser.cpp`	expr／term／factor；結尾必須 END
C-like token、字串、註解	03 `lexer.cpp`	keywords／ops 清單；最長匹配
作業：NUM／PLUS…RPR 切 token	09 `scanner.cpp`	串流讀字元；0 開頭只吃一位
Scanner 切 ID／STRLIT／LBR／RBR／DOT	26 `token_scanner.cpp`	串流；STRLIT 讀到下一個引號
`++ += -- -= -> ** ==` 運算子	27 `op_scanner.cpp`	`cin.peek()` 看下一字元，符合才 get
keyword＋INUM／FNUM 宣告句	16 `decl_scanner.cpp`	先讀完整字再查 keyword；FNUM 點後要有數字
比較運算子＋註解＋錯誤輸入	17 `compare_scanner.cpp`	兩字元優先；錯誤時只印 invalid input
小數／科學記號 NUM（12e）	25 `sci_scanner.cpp`	每段整段成立才吃；退回最後接受位置
作業：ID／STRLIT／DOT 遞迴下降	10 `parser.cpp`	合法才重掃輸出；λ 只在 RBR／END 前
呼叫鏈 `a[0].b(c(1),"x");`	19 `call_parser.cpp`	tail 迴圈；args 逗號；SEMI 結尾
JSON 子集	20 `json_parser.cpp`	value 看第一個 token；`[1,]` 不合法
if／else／while／`{ }` 敘述	23 `stmt_parser.cpp`	stmts 看 FIRST；else 接最近的 if
ac 程式（Fig 2.1）只判語法	28 `ac_parser.cpp`	Dcls／Stmts 兩個 while；結尾 END
RE 本身的語法（`| * ( )`）	29 `re_parser.cpp`	串接看 FIRST(factor)={CHAR, LPR}
布林式 `or and not == < >`	30 `bool_parser.cpp`	or／and 兩層 while；not 遞迴
給 RE，產 NFA（再接 04／13）	24 `re_to_nfa.cpp`	第一行 RE；`| * + ? ( )`，#=λ
給 NFA，判字串	04 `nfa_check.cpp`	# 是 ε；ACCEPT／REJECT
給 NFA，產 DFA 表／集合追蹤	13 `nfa_to_dfa.cpp`	`--table --trace`，預設 `--check`
HW5(a)／(d) 字串辨識	05 `hw5_check.cpp`	第 6 行 `HW5A` true／false
HW5 原 NFA → subset DFA 表	15 `automata_homework.cpp`	`5a --table`／`5d --table`
ac → dc（含型別檢查）	06 `ac_dc.cpp`	只產 dc，不印數值
ac token／parse tree／AST／摺疊	14 `ac_homework.cpp`	`--tokens --parse-tree --ast --typed-ast --check --dc [--fold]`
Stack 指令執行	07 `stack_vm.cpp`	PRINT 會 pop；先 pop 右 b 再左 a
FIRST／FOLLOW／LL(1) 表	08 `ll1.cpp`	預設表格；`--check` 判 token 序列
消左遞迴／left factoring／產 RD parser	22 `rd_gen.cpp`	預設印可編譯 C++；`--grammar` 可接 08

編譯、執行、比對

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror x.cpp -o main
./main < x.in | diff -u x.expected -
```

一次只編一份（每份都有 main）。不用 `windows.h conio.h getch gets system("pause")`。judge 不傳參數 → 把預設模式改成題目要的。

讀題先圈五件事

1. **案例界線**：首行 T／每行一案／整份 EOF 一案。
2. **錯誤時**：只印 invalid input？印 ERROR 繼續？exit 1？
3. **輸出格式**：`種類 原字串`，還是只有種類（ScannerTest 只有 NUM 帶值）。
4. **空白／換行**：可出現在 token 間？STRLIT 可跨行？空輸入合法嗎？
5. **大小寫、編號**：keyword／tag 分大小寫？state 從 0 或 1？

CRLF 與 getline

```
while (getline(cin, line)) {
    if (!line.empty() && line.back() == '\r') line.pop_back();
    // 一案；空行是否略過看題意
}
```

`cin >> T` 後要先 `getline` 吃掉該行剩餘。空行是合法案例時不可用 `cin >> ws`。整份讀入用 `isspace` 跳空白，CR 自動被跳過。

「錯誤時不可輸出任何 token」

兩種做法擇一：**(a)** 先 parse 完，合法才 `pos = 0` 重掃輸出（10、18–21）；**(b)** 輸出先存 `ostringstream`，全部成功才 `cout << out.str()`（03、07、17）。

2｜程式骨架：Scanner + Recursive Descent（照抄改 token）

A. 整份讀入版 Scanner（Parser 題通用）

```
enum Token { NUM, ID, PLUS, LPR, RPR, END, INVALID };
string src; size_t pos = 0, start = 0; Token token;
// [start, pos) 是目前 token 的原字串
void next_token() {
    while (pos < src.size() && isspace((unsigned char)src[pos])) ++pos;
    start = pos;
    if (pos == src.size()) { token = END; return; }
    unsigned char c = src[pos++];
    if (isdigit(c)) {                  // NUM -> 0 | [1-9][0-9]*
        if (c != '0') while (pos < src.size() && isdigit((unsigned char)src[pos])) ++pos;
        token = NUM;
    } else if (isalpha(c) || c == '_') {   // ID
        while (pos < src.size() && (isalnum((unsigned char)src[pos]) || src[pos] == '_')) ++pos;
        token = ID;
    } else if (c == '+') token = PLUS;
    else if (c == '(') token = LPR;
    else if (c == ')') token = RPR;
    else token = INVALID;
}
bool match(Token t) { if (token != t) return false; next_token(); return true; }
```

B. 常見 token 片段（插進 next_token）

```
// keyword：先讀完整個 ID，再比對（intx 是 ID）
string w = src.substr(start, pos - start);
token = w == "int" ? INT : w == "print" ? PRINT : ID;

// 兩字元運算子：先看下一字元（longest match）
else if (c == '<') { if (pos < src.size() && src[pos] == '=') { ++pos; token = LE; } else token = LT; }

// STRLIT "[^"]*"：找不到結尾引號 = INVALID
else if (c == '"') { size_t e = src.find('"', pos);
    token = e == string::npos ? INVALID : STRLIT;
    pos = e == string::npos ? src.size() : e + 1; }

// FNUM [0-9]+\.[0-9]+：點後至少一位才吃點
if (pos + 1 < src.size() && src[pos] == '.' && isdigit((unsigned char)src[pos + 1])) { ... }
```

註解放在跳空白的迴圈裡：

```
for (;;) {
    while (pos < src.size() && isspace((unsigned char)src[pos])) ++pos;
    if (src.compare(pos, 2, "//") == 0) { pos = src.find('\n', pos); if (pos == string::npos) pos =
src.size(); }
    else if (src.compare(pos, 2, "/*") == 0) { size_t e = src.find("*/", pos + 2);
        if (e == string::npos) { pos = src.size(); token = INVALID; return; } pos = e + 2; }
    else break;
}
```

C. 串流版 Scanner（ScannerTest 題，不需重掃）

```
int c = cin.get();
while (c != EOF) {
    if (isspace(c)) { c = cin.get(); continue; }
    if (isdigit(c)) {
        string num(1, char(c)); c = cin.get();
        if (num != "0") while (isdigit(c)) { num += char(c); c = cin.get(); }
        cout << "NUM " << num << '\n';
        continue;            // c 已是下一字元，不可再 get
    }
    switch (c) { case '+': cout << "PLUS\n"; break; /* ... */ default: return 1; }
    c = cin.get();
}
```

D. main：先驗證、合法才重掃輸出

```
int main() {
    const char* names[] = {"NUM", "ID", "PLUS", "LPR", "RPR"};   // 與 enum 同序
    src.assign(istreambuf_iterator<char>(cin), {});
    next_token();
    if (!program() || token != END) { cout << "invalid input\n"; return 0; }
    pos = 0;
    for (next_token(); token != END; next_token())
        cout << names[token] << ' ' << src.substr(start, pos - start) << '\n';
}
```

> `token != END` 一定要查：擋下 `1+2)`、`(a)` 這類「前段合法、後面多東西」。

E. 文法 → 程式 對照表

產生式	寫法
`A → x B y`	`return match(x) && B() && match(y);`
`A → α | β`	依目前 token 屬於哪條的 PREDICT 選分支；都不是 → `false`
`A → … | λ`	λ 分支：`return token ∈ FOLLOW(A)`（不吃 token）
`T → op F T | λ`（tail）	`while (token == op) { next_token(); if (!F()) return false; } return true;`
`S → X S | λ`（list）	`while (token ∈ FIRST(X)) if (!X()) return false;` 之後由呼叫端查 FOLLOW／END
`L → X (COMMA X)* | λ`	`if (token == RBR) return true; if (!X()) return false; while (token == COMMA) { next_token(); if (!X()) return false; }`
左遞迴 `E → E + T | T`	改 `E → T E'`、`E' → + T E' | λ` → while 迴圈
共同前綴 `S → a b | a c`	left factoring：`S → a S'`、`S' → b | c`
語意規則（tag 名相同）	進入時記下名稱，遞迴後再比對：`string n = lexeme(); match(OPEN); content(); return token == CLOSE && lexeme() == n && match(CLOSE);`

F. 優先序分層（低 → 高）與三種輸出

```
expr  → term  { (+|-) term }      // 左結合：while
term  → unary { (*|/) unary }
unary → (+|-) unary | factor      // 一元：遞迴
factor→ NUM | ID | ( expr )
power → factor [ ^ power ]        // 右結合：遞迴而非 while
```

```
double expr() {                     // 求值
    double v = term();
    while (token == PLUS || token == MINUS) {
        Token op = token; next_token();
        double r = term();
        v = op == PLUS ? v + r : v - r;   // 先左後右：100-30-1=69
    }
    return v;
}
// postfix：運算元先輸出，迴圈內在 term() 之後才 out << "+ "
// AST 字串：e = "(" + op + " " + e + " " + r + ")" → (+ a (* b c))
```

G. Stack 配對（HTML／括號）

```
vector<string> st;
// START tag: st.push_back(name);  void／<x/>：不入 stack
// END tag:   if (st.empty() || st.back() != name) invalid; st.pop_back();
// EOF:       valid = st.empty();
```

數量相同 ≠ 巢狀正確（<a>）。引號內的 > 不結束 tag；註解找 `-->`；最大深度 = push 後取 `max(deepest, st.size())`。

H. NFA 模擬與子集構造

```
vector<vector<int>> eps;            // eps[q]：ε 後繼
vector<map<char, vector<int>>> delta; // delta[q][c]
set<int> closure(set<int> s) {
    vector<int> todo(s.begin(), s.end());
    while (!todo.empty()) {
        int q = todo.back(); todo.pop_back();
        for (int r : eps[q]) if (s.insert(r).second) todo.push_back(r);  // 新的才入隊：ε 環不會卡死
    }
    return s;
}
set<int> move(const set<int>& s, char c) {
    set<int> m; for (int q : s) for (int r : delta[q][c]) m.insert(r); return m;
}
// 判字串：cur = closure({start}); 每字元 cur = closure(move(cur, c)); 讀完才看 cur ∩ F
// 子集構造：map<set<int>, int> id; vector<set<int>> D{closure({start})};
// for (i = 0; i < D.size(); ++i) for c in Σ: T = closure(move(D[i], c));
//     沒見過就 id[T] = D.size(), D.push_back(T); trans[i][c] = id[T];
```

I. 固定 DFA：table-driven

```
int s = 0;                        // -1 = dead
for (char c : word) s = s < 0 ? -1 : next_state(s, c);
cout << (s == FINAL ? "ACCEPT\n" : "REJECT\n");   // 讀完才判
```

中途進 final 不能提早接受；字母表外的字元直接到 dead。

3｜文法、FIRST／FOLLOW、LL(1)、AST

算法（08 ll1.cpp 實作同一套）

- **FIRST(α)**：α 第一個 terminal；若第一個符號可為 λ，繼續看下一個；全部可為 λ 則含 λ。
- **FOLLOW(A)**：start 含 `$`；`B → α A β` 把 FIRST(β)−λ 加入 FOLLOW(A)；β 可為 λ（或 A 在尾端）時再加 FOLLOW(B)。反覆到沒有變化。
- **PREDICT(A → α)** = FIRST(α) − {λ}；若 α ⇒* λ，再聯集 FOLLOW(A)。
- **LL(1)**：同一 nonterminal 的各條 PREDICT 兩兩不相交。表格 M[A, t] 一格多條 = conflict。
- Predictive stack：`$ S` 入棧；頂端 terminal 要等於 lookahead；nonterminal 查表後 RHS **反序** push（`S → a S` 先 push S 再 push a）。

例：`S → a S | λ`：FIRST(S)={a, λ}、FOLLOW(S)={$}、PREDICT(S→λ)={$}。`A → B C` 且 C 可為 λ：FOLLOW(A) ⊇ FOLLOW(B)。

ac 文法（Ch2 Fig 2.1）與 PREDICT

#	Production	PREDICT
1	Prog → Dcls Stmts $	floatdcl intdcl id print $
2	Dcls → Dcl Dcls	floatdcl intdcl
3	Dcls → λ	id print $
4	Dcl → floatdcl id	floatdcl
5	Dcl → intdcl id	intdcl
6	Stmts → Stmt Stmts	id print
7	Stmts → λ	$
8	Stmt → id assign Val Expr	id
9	Stmt → print id	print
10	Expr → plus Val Expr	plus
11	Expr → minus Val Expr	minus
12	Expr → λ	id print $
13–15	Val → id | inum | fnum	id ／ inum ／ fnum

ac token：f／i／p 是 floatdcl／intdcl／print；id 為單一小寫字母（扣 f i p，共 23 個）；inum `[0-9]+`；fnum `[0-9]+\.[0-9]+`。b2 → id(b) inum(2)；`-5` → minus inum。

Parse tree／AST 要印什麼（14 ac_homework）

	保留	省略
Parse tree	所有 nonterminal、terminal、λ、$；葉由左到右 = token 串	改寫先後
AST	運算子、運算元、宣告／敘述順序	Dcls／Stmts／Val／Expr 包裝、括號、λ、$
Decorated AST	AST＋型別，可插 int2float	只有要求型別時才加

`x=a+b*c` → `assign(x, plus(a, times(b,c)))`；`x-30-1` 左結合 → `minus(minus(x,30),1)`（69，右結合得 71）。

4｜Scanner 規則、NFA → DFA 核對表

Scanner 規則

- **Maximal munch**：取最長合法前綴；同長才比規則先後（keyword 規則在 ID 前）。`float`→FLOAT、`floatx`→ID、`>>=` 一個 token。
- 長候選失敗要退回**最後接受位置**（只有 a、abc 兩種 token，輸入 abx → 回 a，從 b 續掃）。
- 整串辨識（recognize）≠ 切 token（tokenize）：`12e` 不是完整 NUM，但 scanner 可能切 NUM(12) ID(e)。
- 負號是 MINUS，不併進數字；`007`：ac 是一個 inum，ScannerTest 是 NUM 0 ×3。
- quoted literal 內的 `//` 不是註解。

輸入	結果（依該題 RE）
`012`	NUM 0、NUM 12（NUM = 0 | [1-9][0-9]*）
`intx／int x`	ID intx ／ INT、ID x
`a<=b／a<b>c`	ID LE ID ／ ID LT ID GT ID
`3.／.5`	INUM 3 後 . 非法 ／ . 非法
`honeycone=5+10*num;`	8 tokens：ID ASSIGN NUM PLUS NUM TIMES ID SEMI

常用 token RE

教材 identifier：`L(L|D)*(_(L|D)+)*`（拒 `_foo`、`foo_`、`foo__bar`）。題目給 `[A-Za-z_][A-Za-z0-9_]*` 就照題目。
科學記號（HW3 補正）：`[0-9]+(\.[0-9]+)?(e[+-]?[0-9]+)?`；拒 `12e`、`1.2e-`、`.1`、`10.`、`3.e2`。教材解答漏了 e 可省、+ 號、點前多位。

NFA → DFA（子集構造）

> $S_0 = \text{close}(\{q_0\})$；$\delta_D(S,c) = \text{close}(\text{move}(S,c))$；S 接受 ⇔ $S \cap F \neq \varnothing$

1. close 包含自己，沿 λ 走到固定點（不是只走一步）。
2. 每個新集合依字母表順序算每個字元；相同集合沿用名稱（順序無關）。
3. ∅ 是拒絕 sink，完整表要列（若可達）。
4. n 個 NFA state 最多 2^n 個子集，只列可達的。
5. 測例：λ、最短接受、近似錯例（讀完才判）。

下列表格可當 13 nfa_to_dfa／15 automata_homework 的輸出核對。

HW5(d)：(bc)*d　NFA start=1、F={6}

NFA	b	c	d	λ
→1				2,5
2	3			
3		4		
4				2,5
5			6	
★6				

DFA	集合	b	c	d
→A	{1,2,5}	B	X	D
B	{3}	X	C	X
C	{2,4,5}	B	X	D
★D	{6}	X	X	X
X	∅	X	X	X

bcd：A→B→C→D ✓；bc 停 C ✗；bcdd 到 X ✗。05 hw5_check 的 state：0 起始、1 剛讀 b、2 剛讀 bc、3 接受、-1 dead。

HW5(a)：(a|(bc)*d)+　NFA start=1、F={12}

DFA	NFA 集合	a	b	c	d
→A	{1,2,3,6,7,10}	B	C	X	D
★B	{2,3,4,5,6,7,10,12}	B	C	X	D
C	{8}	X	X	E	X
★D	{2,3,5,6,7,10,11,12}	B	C	X	D
E	{7,9,10}	X	C	X	D
X	∅	X	X	X	X

一塊 = a 或 (bc)*d，至少一塊。d、a、ad、bcd 接受；λ、bc、bd 拒絕。沒有 start→12 的零次旁路。

Fig 3.24：start=1、F={5}

NFA：1—a→3、1—b→1、1—λ→2、2—a→{4,5}、3—b→4、4—a,b→5。

DFA	集合	a	b
→A	{1,2}	B	A
★B	{3,4,5}	D	C
★C	{4,5}	D	D
★D	{5}	X	X
X	∅	X	X

起始閉包是 {1,2}，不含 3（教材 p.47 筆誤）。

(a|b)*abb（11-state NFA，start=0、F={10}）

DFA	集合	a	b
→A	{0,1,2,4,7}	B	C
B	{1,2,3,4,6,7,8}	B	D
C	{1,2,4,5,6,7}	B	C
D	{1,2,4,5,6,7,9}	B	E
★E	{1,2,4,5,6,7,10}	B	C

無可達 ∅。abb、aabb 接受；abba 停 B ✗。

5｜語意、型別、dc 產碼

Symbol table 與型別（ac）

- 表：23 個 id → int／float／null。EnterSymbol：已有型別 → duplicate；Lookup null → 未宣告。表存型別，不存值。
- 型別 bottom-up：葉 inum=int、fnum=float、id 查表；int±float → int 側包 `int2float`，結果 float。
- float ← int：RHS 包 int2float；int ← float：拒絕（本地實作；`i x x=1e0` 也拒）。

輸入（各自獨立）	錯誤層級
`i a a=@`	Lexical：@ 不符合任何 token
`i a a=+5／i a p 5／i a a=5 f b`	Syntax：違反文法
`i a f a／f b b=c`	Semantic：重複宣告／未宣告
`i a a=3.2`	Semantic：float 不可隱式指定給 int

AST → dc（Fig 2.14）

節點	輸出
inum／fnum	值
id r	`lr`
L op R	code(L) code(R) op（後序）
int2float(E)	code(E) `5 k`
r = E	code(E) `sr 0 k`
p r	`lr p si`
宣告	不產碼

```
f b  i a  a = 5  b = a + 3.2  p b
→ 5 sa 0 k   la 5 k 3.2 + sb 0 k   lb p si      （印 8.2）
```

`sr` pop；`lr` push 副本；`p` 不 pop（所以接 `si` 丟到保留字 i）；`5 k` 是 push 5 再由 k pop 當 scale。減法 `L R -` = L−R：`100 30 - 1 -`=69；`100 30 1 - -`=71。科學記號要轉成十進位字串：`123e-22` → `0.00000000000000000123`（不可經 double）。HW2(c) 摺疊後：`100 sx 0 k / lx 31 + sx 0 k / lx p si`。

Stack VM（07）

二元：先 pop 右 b、再 pop 左 a，算 a op b（`PUSH 8 PUSH 2 SUB` = 6）。STORE、PRINT 都 pop；要保留就改 peek。underflow／未定義／零除 → stdout 空、exit 1。

6｜各程式卡片：I/O、核心、陷阱、常見改法（行號 → 改成）

01 HTML 標籤配對 `html_check.cpp`

整份 EOF 一案 → VALID／INVALID。名稱不分大小寫；void、`<x/>` 不入 stack；script／style 讀到對應 END。

陷阱：`<p a=x/>` 沒關（/ 屬於值），`<p a=x />` 有關；`<!-- unclosed` INVALID；空輸入 VALID。

- 127 YES/NO：`cout << (valid ? "YES\n" : "NO\n");`
- 121–127 一行一案：`while (getline(cin, s)) { p = 0; …check… }`；首行 T 版先讀 T 再 getline 吃行尾
- 32–33 XML 式（br 也要關）：`const set<string> empty{};`
- 最大深度：11 後加 `size_t deepest = 0;`；101 push 後更新；127 印 deepest

02 遞迴下降運算式 `expression.cpp`

每行一式；eval 印 15 位有效數字；postfix 一元用 pos／neg；ast 印前序括號。非法、零除 → stdout 空、exit 1。巢狀上限 256。

陷阱：空白行是非法式；`2(3)` 不是隱含乘法；`-2*-3`=6。

- 12 `const string MODE = "postfix";`（或 `"ast"`）
- 106 固定兩位：`out << fixed << setprecision(2);`
- 51 後禁止變數：`if (id) fail();`
- 85 整數除法向零：`a.number = trunc(a.number / b.number);`

03 C-like Scanner `lexer.cpp`

每行 `種類 "lexeme"`，最後 EOF。KW／ID／NUM／STRING／CHAR／OP／PUNCT；註解 `//`、`/* */`。錯誤 → stdout 空、exit 1。

- 26 keywords 清單、28 ops 清單（長的放前面）
- 90–92 不加引號：`output << kind << ' ' << s.substr(start, p - start) << '\n';`
- 95 不印 EOF：刪除該行
- 47 keyword 全當 ID：`kind = "ID";`

04 NFA 判定字串 `nfa_check.cpp`

輸入 `n m start k`、k 個 final、alphabet、m 條 `from sym to`（#=ε），之後每行一字串（空行 = λ）→ ACCEPT／REJECT。

- 31／40 state 從 1：`state >= 1 && state <= n`、`edges.resize(n + 1);`
- 20／37／44 ε 改 `@`：三處 `'#'` → `'@'`
- 68 YES/NO；53 去頭尾空白；55／63 後插入印集合

05 HW5 固定 DFA `hw5_check.cpp`

每行一字串 → ACCEPT／REJECT；大小寫敏感。

- 6 `const bool HW5A = false;` 改解 (d)
- 30 外層 + 改 *：`(state == 3 || word.empty())`
- 13 後不分大小寫：`for (char& c : word) if (c >= 'A' && c <= 'Z') c += 'a' - 'A';`
- 9 後 `int case_no = 0;`，30 印 `"Case " << ++case_no << ": "`

06 ac → dc `ac_dc.cpp`

整份 EOF；`f`／`float`、`i`／`int`；指數只收小寫 e。合法 → dc code；非法 → stdout 空、exit 1。

- 35／57／98 接受 E：三處 `'e'` → `"eE"`／`".eE"`
- 128／131／136 scale 改 10：`"10 k\n"`
- 138 不要 `0 k`：`out << 's' << target << "\n";`
- 140 只印 OK：`cout << "OK\n";`；43–45 只收 i／f 短宣告

07 Stack VM `stack_vm.cpp`

每行一指令 PUSH／LOAD／STORE／ADD／SUB／MUL／DIV／NEG／DUP／PRINT；空行忽略；15 位有效數字。

- 57 PRINT 不 pop：`const auto value = stack.back();`（先查 empty）
- 57 改名 OUT：`op == "OUT"`
- 32 兩位小數：`output << fixed << setprecision(2);`
- 41 後不分大小寫：`for (char& c : op) c = toupper((unsigned char)c);`

08 LL(1) `ll1.cpp`

P 行 `A -> X Y`（# 單獨 = λ）；印 FIRST／FOLLOW／SELECT／M[A,t]／LL1 YES|NO；`--check` 每行 token 序列 → ACCEPT／REJECT。

- 38 judge 不傳參數也判字串：`argc == 1 ? "--check" : argv[1]`
- 130 YES/NO；96–106 只留需要的輸出（106 是 LL1 YES/NO）
- 指定 S 為 start：71 `follow["S"]`、115 `stack{"$", "S"}`

22 文法改寫＋RD 產生 `rd_gen.cpp`

輸入同 08。自動消左遞迴（直接＋間接）→ left factoring → 算 SELECT → 印完整 C++：每個 nonterminal 一個 `parse_X()`，每條規則一行 `if (SELECT) return …;`，`tok()`／`match()` 以字串比對，main 每行判 ACCEPT／REJECT。新 nonterminal 叫 `A'`（函式 `parse_A_p`）。SELECT 重疊 → stderr 印 CONFLICT、exit 1、不產碼。

```
./main --grammar < g | ../08-LL1/main   # 看表
./main < g > p.cpp                       # 產 parser
```

套進考題：`tok() == "num"` → `token == NUM`、`match("num")` → `match(NUM)`，函式結構不動；main 改成第 2 節 D。不處理 nullable 前綴的隱藏左遞迴、dangling else。

09 作業 ScannerTest `scanner.cpp`

NUM／PLUS／MINUS／MUL／DIV／LPR／RPR，每行一個，只有 NUM 帶值。輸入保證無錯。

- 17 前導 0 也連吃：`while (isdigit(c)) { num += char(c); c = cin.get(); }`
- 28 加 `case '%': cout << "MOD\n"; break;`
- 30 非法字元印 ERROR 繼續：`default: cout << "ERROR " << char(c) << '\n'; break;`

10 作業 ParserTest `parser.cpp`

stmt → primary | STRLIT | λ；primary_tail → DOT ID … | LBR stmt RBR … | λ。合法印 `種類 原字串`，否則只印 invalid input；空輸入合法。

陷阱：`a.`、`(a)`、`"abc`（未閉）都 invalid；λ 只能接 RBR／END。

- 括號內逗號：7 enum 加 COMMA、31 加 `','`、54 `while (token == COMMA) { next_token(); if (!stmt()) return false; }`、62 names 加 "COMMA"
- YES/NO：刪 62（names），73–80 換成 `cout << (valid && token == END ? "YES\n" : "NO\n");`
- 26 STRLIT 不跨行：`find_first_of("\"\n", pos)`，遇 `\n` 視為 INVALID

11–15 完整版（要樹、表、追蹤時才用）

11 HTML token／tree／text；12 迭代算式（無深度上限）；13 任意 NFA → DFA 表與 trace；14 ac 全套（parse tree、AST、typed AST、fold、dc）；15 HW5 原 NFA → subset DFA 表。用法見第 1 節表格與各資料夾 README。

16–21、23、25–30 模擬題（README 即題目）

- 16 宣告 Scanner：INT／FLOAT／PRINT 先讀完整字；`007` → 三個 INUM；`3.` 非法（exit 1）。
- 17 比較運算子：LE 先於 LT；`/* 未閉`、單獨 `!`／`/` → invalid input。
- 18 四則 Parser：`1+*2`、`(1+2`、`012` invalid。
- 19 呼叫鏈：`f()` 合法、`f(,1)`、缺 `;`、`a[b]` invalid。
- 20 JSON：`[1,2,]`、`{"a" 1}`、`[01]`、`True` invalid。
- 21 HTML tags：`<b><i>x</b></i>`、`a > b`、未關 invalid；TEXT 壓縮空白。
- 23 敘述：`(!match(ELSE) || stmt())` 解 dangling else；`else x=1;`、`{` 未關 invalid；`iff` 是 ID。
- 25 科學記號：`12e` → NUM 12、ID e；`1.2e-` → NUM 1.2、ID e、MINUS；`3.`、`.5` invalid；`12E3` → NUM 12、ID E3。
- 26 符號 Scanner：輸出 `ID x`、`STRLIT "a b"`，其他只印種類；不查文法，`(a.` 照切。
- 27 複合運算子：`+++` → INC PLUS；`**=` → POW ASSIGN；`012` → NUM 0、NUM 12。
- 28 ac：`ab = 1`（兩個 ID）、`i a a = +5`、`i a p 5` invalid；不做型別檢查。
- 29 RE：`a**` 合法；`a||b`、`()`、空輸入 invalid。
- 30 布林：`order == 1` 合法（order 是 ID）；`a = 1`、`not a` invalid。

24 RE → NFA（Thompson） `re_to_nfa.cpp`

第一行 RE，輸出 04／13 的 NFA 格式，之後的測試字串原樣接上：

```
./main < x.in | ../04-NFA判定字串/main            # ACCEPT／REJECT
./main < x.in | ../13-NFA轉DFA完整版/main --table  # DFA 表
```

A* 加零次旁路與重複邊、A+ 只有重複邊、A? 只有旁路。只有 # 的 RE 字母表為空，04／13 會拒收。

7｜出錯時依症狀找

症狀	先查
多／少一行	T 與 EOF 混用；空行被跳過；最後換行
invalid 卻印了 token	沒先驗證就輸出；改用重掃或 ostringstream
合法輸入被判 invalid	λ 分支的 FOLLOW 漏了 END／RBR；match 後忘記 next_token
非法輸入被判合法	main 沒查 `token == END`；λ 分支直接 return true 吞掉怪 token
token 少一個字元	串流版多讀一字元後又 `get()`；或 pos 沒退
運算結果不同	結合性（69 vs 71）；pop 順序（先 b 後 a）；整數除法
小數格式不同	`fixed`＝小數位數；不加 fixed＝有效位數
NFA 判錯	初始沒 closure；move 後沒 closure；中途就接受
Runtime Error	空 stack pop／越界／`isspace` 傳負 char（先轉 unsigned char）／深遞迴
TLE	迴圈沒推進 pos；ε 環重複入隊；字串拼接 $O(n^2)$

提交前測例

1. 題目全部樣例，逐 byte 比對（空白、最後換行）。
2. 空輸入、最短合法、最短非法。
3. 多層巢狀、交錯關閉、未閉合（括號、引號、註解）。
4. 最長匹配邊界：`<=`、`intx`、`012`、`3.`。
5. 同級左結合、負數、零值、除以零。
6. 多案連續：每案重設 pos、stack、計數器。
7. LF 與 CRLF 各一次。

> 原則：先讓原檔通過樣例，一次只改一處，每改一處重跑。行號位移時搜尋本頁列出的程式片段。